

strip 스펙트로그램 띠

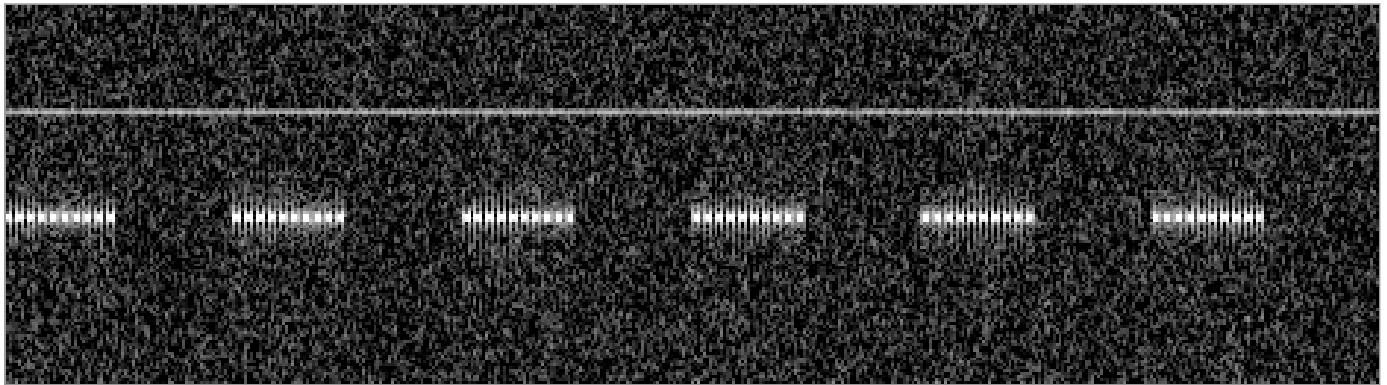
GUI(FFT 256 · Hamming · 0~fs/2)와 같은 표시로 같은 파일을 그린다 — 디코드가 같은지 눈으로 판정하는 결정 실험

코드 62줄 · 총 3쪽 | 2026-08-18 · 3cab2a6+수정중 | 출력은 PNG 파일 — 아무 이미지 뷰어로 연다 (matplotlib 불필요)

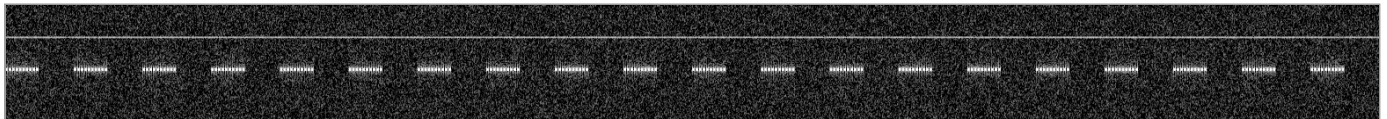
- ① 이 코드를 **signus/** 옆에 **strip.py** 로 저장한다.
- ② **python3 strip.py kat** — 출력 한 줄을 아래와 글자까지 비교하고, 생성된 **strip-kat.png** 를 열어 아래 기대 그림과 모양을 비교한다 (다른 필사 오타).
- ③ **python3 strip.py <캡처파일> — <캡처파일>.png** 가 생긴다. GUI 로 같은 파일을 연 화면과 나란히 비교한다.
- ④ 회신 세 가지: (가) strip 출력 한 줄(#코드까지) (나) PNG 가 아래 두 기준 그림 중 어느 부류인지 한 마디 (다) GUI 가 표시하는 파일 길이(초) — 출력의 **s** 값과 다른 샘플 해석(비트수/채널)이 다른 것이다.

```
strip kat n60000 f10000 s6.0 px467x129 lo-43 rg27 #wm8w
```

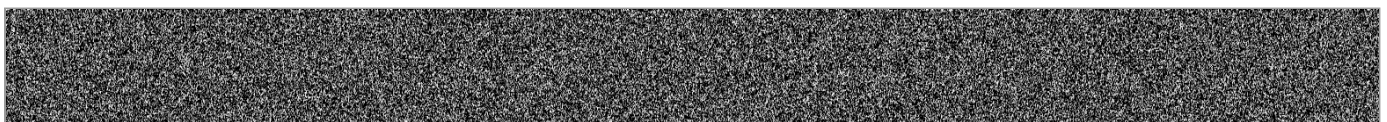
kat 기대 그림 — 장비의 strip-kat.png 가 이와 같아야 한다



버스트가 있는 캡처의 예 — GUI 에 신호가 보인다면 이 부류



잡음뿐인 캡처 — 지금 signus 가 읽은 결과는 이 모습을 예언한다



```

1  """strip.py - 캡처를 GUI 와 같은 방식(FFT 256 · Hamming · 50% 겹침 · 0~fs/2)의 흑백
2  스펙트로그램 띠 PNG 로 그린다 (장비 필사용, signus/ 옆에 저장).
3      python3 strip.py kat          # 자기검증: strip-kat.png 생성 + 출력 줄을 표지와 대조
4      python3 strip.py <캡처파일>   # <캡처파일>.png 생성 - GUI 와 나란히 비교
5  """
6  import struct
7  import sys
8  import zlib
9
10 import numpy as np
11 from scipy.signal import stft
12
13 from signus import sigio
14 from signus.cli import check_code
15
16
17 def png_gray(img, path):
18     img = np.clip(img, 0, 255).astype(np.uint8)
19     h, w = img.shape
20
21     def chunk(tag, data):
22         c = tag + data
23         return struct.pack(">I", len(data)) + c + struct.pack(">I", zlib.crc32(c))
24
25     raw = b"".join(b"\x00" + img[r].tobytes() for r in range(h))
26     with open(path, "wb") as fh:
27         fh.write(b"\x89PNG\r\n\x1a\n"
28                 + chunk(b"IHDR", struct.pack(">IIBBBBB", w, h, 8, 0, 0, 0, 0))
29                 + chunk(b>IDAT", zlib.compress(raw, 6)) + chunk(b"IEND", b""))
30
31
32 if len(sys.argv) < 2:
33     raise SystemExit("사용법: python3 strip.py kat | python3 strip.py <캡처파일>")
34 arg = sys.argv[1]
35 if arg == "kat":
36     fs, n = 10000.0, 60000
37     t = np.arange(n) / fs
38     rng = np.random.default_rng(7)          # 시드 고정 -- numpy 가 재현을 보장한다
39     key = ((np.arange(n) // 250) % 2 == 0) & ((np.arange(n) // 5000) % 2 == 0)
40     x = np.sin(2 * np.pi * 2200 * t) * key + 0.12 * np.sin(2 * np.pi * 3600 * t)
41     x = x + 0.18 * rng.standard_normal(n)
42     name = "strip-kat"
43 else:
44     x, meta = sigio.read(arg)
45     fs = meta.fs
46     x = x.real if np.iscomplexobj(x) else x    # 실수 파형 기준 -- GUI 와 같은 입장
47     name = arg
48 _, _, z = stft(x, fs=fs, window="hamming", nperseg=256, noverlap=128,
49                 return_onesided=True, boundary=None, padded=False)
50 db = 10 * np.log10(np.abs(z) ** 2 + 1e-12)
51 lo = float(np.percentile(db, 25))            # 바닥 근처를 검정으로
52 rg = float(max(10.0, min(35.0, np.percentile(db, 99.5) - lo))) # 자동 대비 -- 고정 35dB 는
53 img = np.clip((db - lo) / rg, 0, 1)[::-1]    # 6dB 급 차이를 씻어내 GUI 와 비교가 안 된다
54 # (아래=0Hz, 위=fs/2 = GUI 의 Y MAX)
55 k = max(1, img.shape[1] // 4000)           # 폭 ~4000픽셀 제한: 열 최대값 폴링이라
56 img = img[:, :img.shape[1] // k * k].reshape(img.shape[0], -1, k).max(2) # 버스트는 남는다
57 png_gray(img * 255, name + ".png")
58 line = (f"strip {'kat' if arg == 'kat' else 'cap'} n{x.size} f{fs:.0f}"
59         + f" s{x.size / fs:.1f} px{img.shape[1]}x{img.shape[0]} lo{round(lo)} rg{round(rg)}")
60 print(f"{line} #{check_code(line)}")

```

```
61 | print(f"{name}.png 저장 - 이미지 뷰어로 열어 GUI 와 나란히 비교. s 값(초)이 GUI 가 보여주는"  
62 |     " 파일 길이와 다르면 샘플 해석(비트수/채널)이 다른 것이다")
```